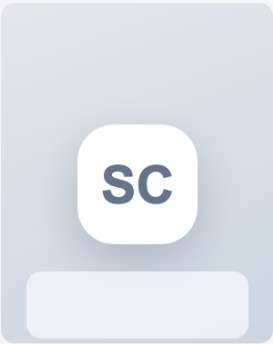


Example Product JD에 맞춘 프론트엔드 포트폴리오



Sample Candidate

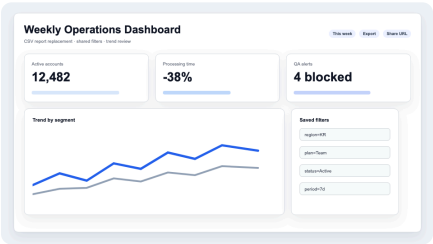
Frontend Engineer

sample@example.com

[github](#)

[portfolio](#)

프로젝트 목록



Sample Dashboard

- 2p 프로젝트 소개, 담당 역할, 대시보드 화면
- 3p 핵심 기술 이슈 - TanStack Query + URL query

Review queue

Application ID	Status	Action
Application #101	Review	Review
Application #102	Waiting	Review
Application #103	Review	Review
Application #104	Waiting	Review
Application #105	Review	Review
Application #106	Waiting	Review

Sample Admin Workflow

- 4p 검토 큐 문제, 설계 결정, 결과

Partner onboarding form

Company name

Manager email

Contract start date

Submit

Sample Form System

- 5p 폼 검증, 컴포넌트 상태 규칙
- 6p JD 연결점과 제출 전 교체 항목



Sample Dashboard

2025.07-2025.08 [React](#) [TypeScript](#) [TanStack Query](#) [Recharts](#)

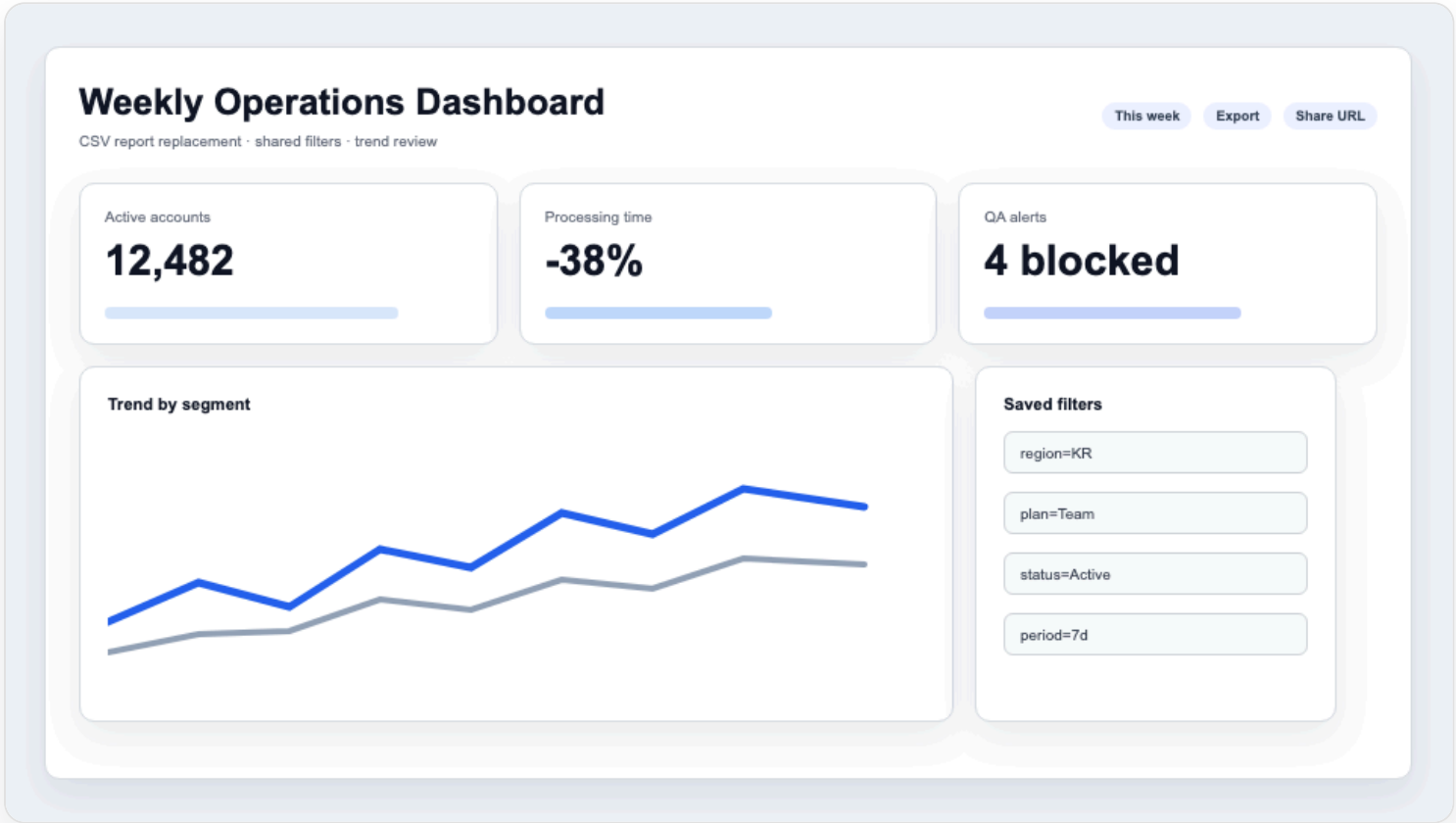
프로젝트 소개

내부 운영자가 여러 CSV를 수동으로 합쳐 보던 주간 지표를 한 화면에서 확인하도록 만든 대시보드입니다.

담당 역할

프론트엔드 리드로 서버 상태, 필터 상태, 차트 시각화 흐름을 설계하고 구현했습니다.

- TanStack Query 기반 서버 상태 분리
- URL query 기반 필터 조건 동기화
- Recharts 기반 주요 지표와 추세 시각화



1. 문제 상황

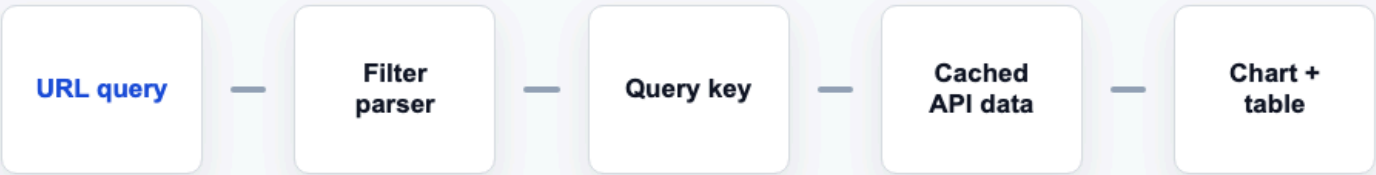
- 운영자가 여러 CSV를 수동으로 합쳐 주간 지표를 확인
- 필터 조건 공유와 같은 조건 재현이 어려움

2. 결과

- 주간 리포트 작성 시간이 2시간에서 30분으로 줄었다는 운영자 피드백 확보
- 필터 조건 공유와 재현이 쉬워졌다는 정성 피드백 확보

TanStack Query + URL query 상태 설계

Reproducible dashboard state



Same link restores the same filters, query cache boundary, and report view after refresh.

1. 문제 상황

- 필터 변경과 데이터 요청이 한 컴포넌트 안에 섞여 변경 영향 범위가 커짐
- 운영자가 동일 조건을 공유해도 화면 상태가 재현되지 않음

2. 행동

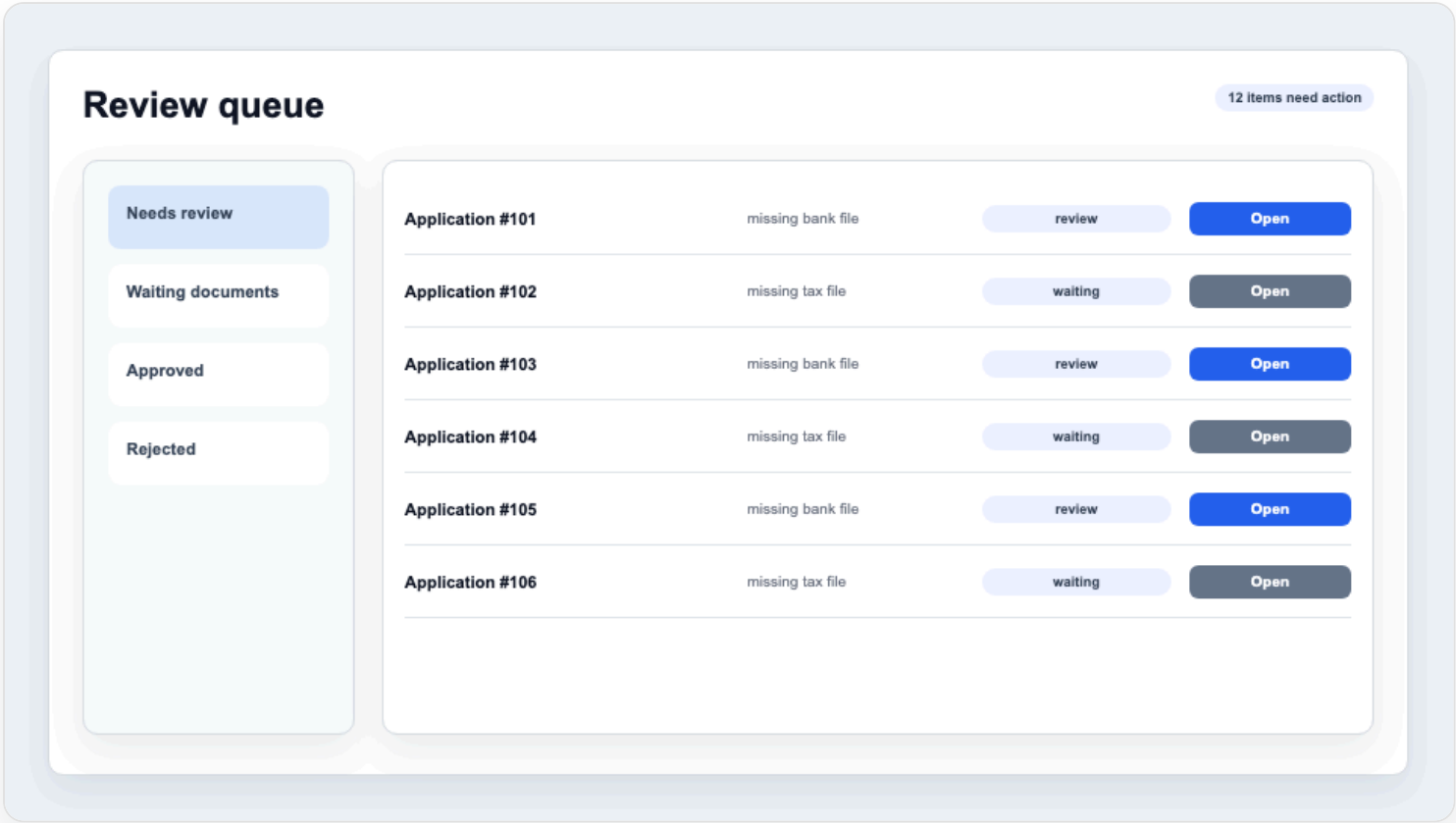
- 서버 데이터 요청과 캐시 관리는 TanStack Query로 이동
- 필터 조건은 URL query로 직렬화하고 화면 진입 시 복원
- 차트와 테이블은 같은 query state를 기준으로 렌더링

3. JD 연결점

- 대시보드 UI, API 연동 구조, 서버 상태 관리, 운영자 흐름 개선 요구와 직접 연결
- 업무용 화면에서 상태 설계 판단을 설명할 수 있는 사례로 사용

Sample Admin Workflow

React · URL query · Table UI · Review workflow



1. 문제 상황

- 검토 담당자가 신청 상태, 누락 서류, 담당자 메모를 여러 화면에서 확인
- 필터 조건 공유가 어려워 스크린샷이나 구두 설명에 의존

2. 행동

- 상태별 탭과 리스트 필터를 URL query로 동기화
- 상세 패널에서 누락 서류, 담당자 메모, 처리 액션을 한 번에 확인하도록 배치
- 액션 버튼의 loading, disabled, error 상태를 명시해 중복 처리 가능성을 낮춤

3. 결과

- 같은 조건의 검토 큐를 팀원에게 링크로 전달할 수 있게 됨
- 화면 이동 없이 누락 서류와 처리 상태를 확인할 수 있다는 정성 피드백 확보

Sample Form System

TypeScript · Zod · Form component · Accessibility

Partner onboarding form

Zod schema · typed API response · error mapping

Company name

Manager email

Enter a valid work email address.

Contract start date

Submit

Save draft

Form component rules

Default

Label

Helper text

focus

keyboard

ARIA

Error

Label

Required field

focus

keyboard

ARIA

Disabled

Label

Helper text

focus

keyboard

ARIA

1. 문제 상황

- 폼마다 입력 상태와 에러 메시지 표현이 달라 QA 피드백이 반복
- 사용자가 어떤 입력을 잘못했는지 화면에서 바로 알기 어려움

2. 행동

- Zod 스키마로 입력 데이터의 구조와 검증 기준을 명시
- label, helper text, error text, disabled state의 우선순위를 정의
- 키보드 접근과 focus ring 동작을 점검해 기본 사용성을 보강

3. 결과

- QA 재현 이슈 5건 중 4건을 사전 검증 단계에서 차단
- 신규 폼 화면에서 상태별 UI 리뷰 기준이 명확해짐

Example Product JD 연결 요약

Dashboard UI

운영자가 지표, 필터, 추세를 한 화면에서 비교하도록 상태와 정보 구조를 설계했습니다.

API State

서버 상태, UI 필터, URL 상태를 분리해 재현 가능한 화면 상태를 만들었습니다.

Form Quality

Zod 스키마와 에러 메시지 매핑으로 입력 단계에서 QA 이슈를 줄였습니다.

Workflow

검토 큐, 상세 패널, 액션 버튼 상태를 나눠 운영자 처리 흐름을 단순화했습니다.

1. 실제 제출 전 교체할 증빙

- 실제 GitHub repository, PR, 이슈 링크
- 실제 대시보드와 폼 화면 스크린샷
- 운영자 피드백 원문과 before/after 측정 방식

2. JD 맞춤 조정 방향

- 대시보드와 API 상태 관리 요구가 강하면 Sample Dashboard를 첫 사례로 유지
- 어드민 운영 효율 요구가 강하면 Sample Admin Workflow를 앞쪽으로 이동
- 입력 품질과 TypeScript 요구가 강하면 Sample Form System의 수치와 PR 근거를 보강